release through gated GitHub Actions to a real OpenShift target -- with rollback rehearsed, not assumed."

MODULE 51

Capstone Security, CI-CD and Deployment

Harden the Angular/Spring Boot API with JWT and deny-by-default RBAC, gate GitHub Actions with SAST/DAST and container scans, and prove a digest-pinned OpenShift deploy with authenticated smoke evidence and a rehearsed rollback.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 51 OVERVIEW

Ship a secured, gated Angular/Spring Boot release to OpenShift, with authenticated and denied-path smoke evidence and a rehearsed rollback.

Feature-complete is not release-complete. This module hardens JWT/RBAC, gates GitHub Actions with security scans, and proves deploy, smoke, and rollback against OpenShift, the real target.

DURATION & LAB

4 Hours Theory

JWT/RBAC hardening, SAST/DAST and container scanning, Terraform/Ansible stages, and OpenShift deployment.

2 Hours Lab

Plan the gated pipeline, the digest-pinned image, the OpenShift deploy, and the smoke/rollback evidence this build proves.

Scenario

Ship a release candidate for Amina (CUS-1001) with authenticated smoke passing and anonymous calls returning 401.

MODULE ARC

Deny By Default

A JWT resource server first -- every endpoint requires a role unless explicitly permitted.

Immutable Images

Digest-pinned, non-root, multi-stage builds -- never deploy a floating tag.

Rollback Rehearsed

A previous-digest rollback is proven on OpenShift before it's ever needed for real.

KEY TAKEAWAY Total time: 4 hours theory + 2 hours lab = 6 hours. No 'deployed' claim without digest identity, authenticated and deny-path smoke evidence, and a rehearsed rollback.

CAPSTONE DELIVERY ARCHITECTURE

A working feature is not a shippable release -- access control, scanning, image provenance, and recovery must be evidenced together, on OpenShift.

Feature-Complete Is Not Release-Complete

Merge/promote freezes until security, delivery, provenance, and recovery are all proven.

Open Endpoints Fail the Course

An open /api/** on the shared OpenShift cluster is a named, non-negotiable security failure.

Immutable or It Didn't Ship

A floating tag with no digest means no one can prove what is actually running.

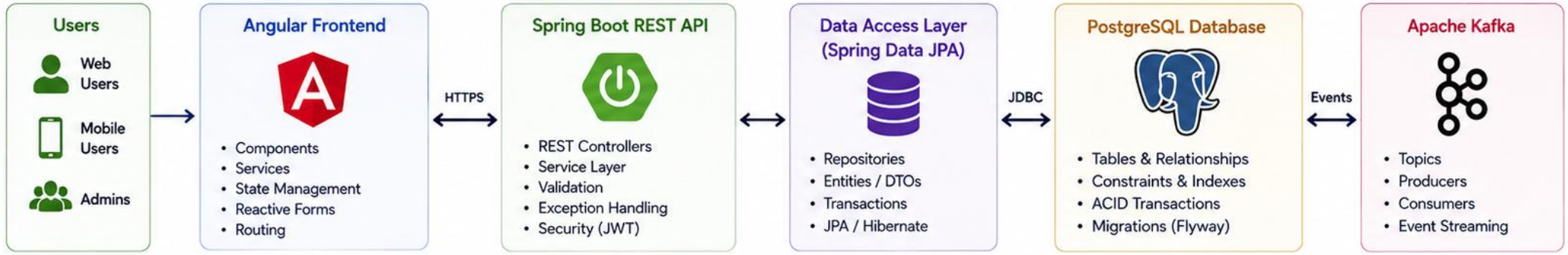
Every Stage Is Evidence

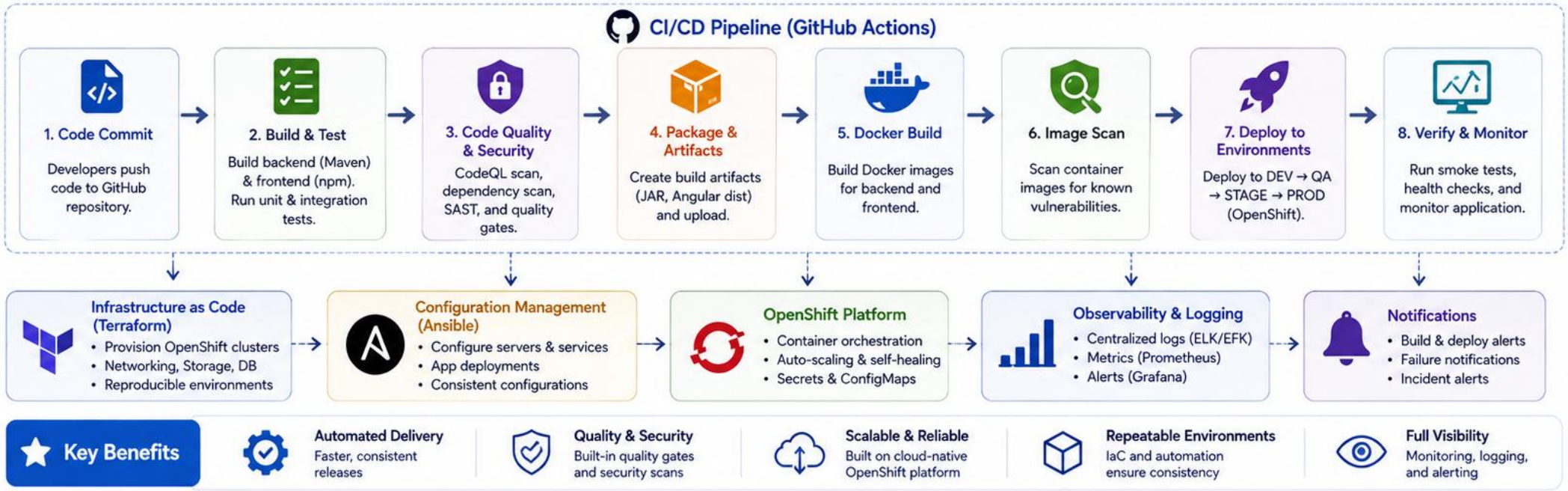
Each pipeline stage from Module 48's plan produces a recorded artifact, not a verbal claim.

KEY TAKEAWAY Floating tags without a digest and a skipped unauthorized smoke test are named, graded failure modes -- not style notes.

1 Capstone Delivery Architecture

End-to-end delivery pipeline from code to production with automated quality, security, and deployment.





GITHUB ACTIONS END-TO-END WORKFLOW

The pipeline gate order is fixed: never deploy an artifact that has not been verified, scanned, and published first.

Stage	Purpose
build	Compile Angular and Spring Boot; produce a build artifact.
verify	Run the full test suite -- backend and Angular must both be green.
scan	Secret, dependency, SAST, DAST, and image scans -- criticals block unless exceptioned.
publish -> deploy (gated)	Push the digest-identified image, then roll out to OpenShift only after every prior stage passes.

PIPELINE HABITS

Secrets Stay Scoped

Registry and OpenShift credentials live in Actions secrets, never in YAML.

Identity Flows Forward

The same verified tag/digest moves stage to stage -- no silent rebuild.

Approvals Where Required

A deploy gate requires an explicit human approval via GitHub Environments.

Reports Preserved

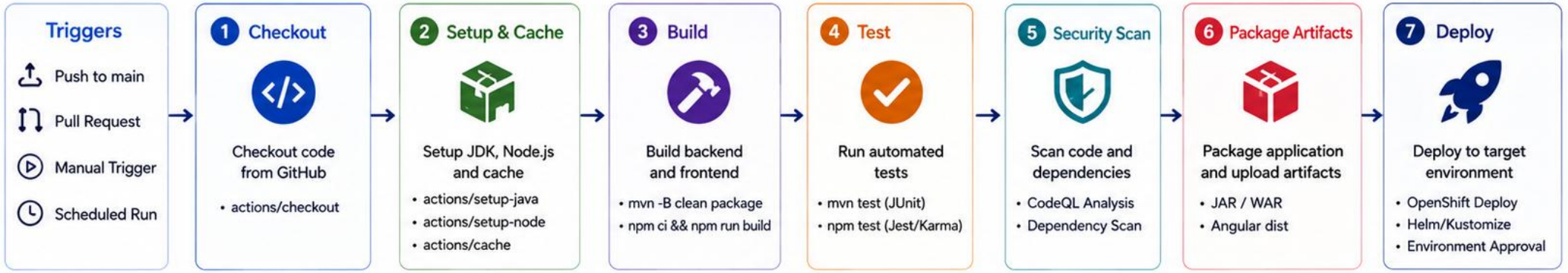
Test and scan reports are saved as pipeline artifacts, not discarded.

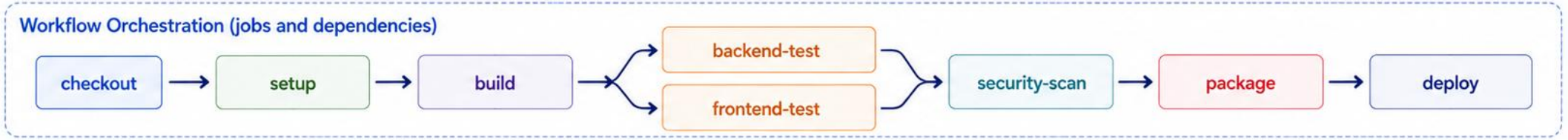
KEY TAKEAWAY A pipeline that deploys before its scan stage completes is not gated -- it is just automated, which is a very different guarantee.

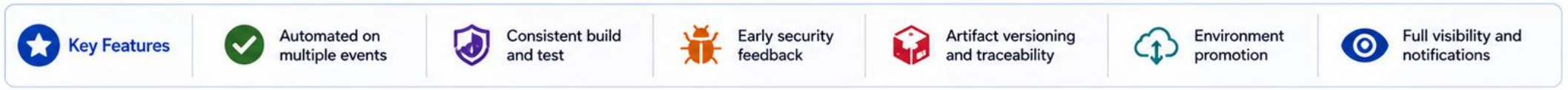
2 GitHub Actions End-to-End Workflow

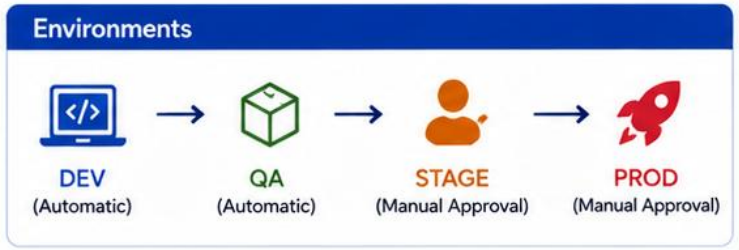
GitHub Actions

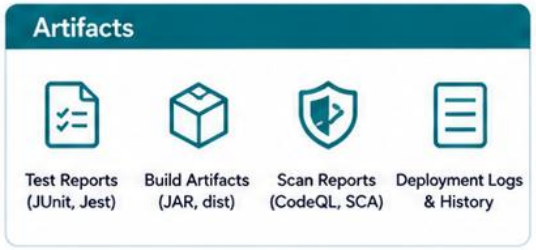
Automate build, test, security, and deployment for the full-stack capstone application.

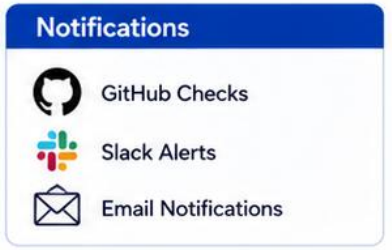












PULL REQUEST QUALITY GATES

Module 48's branch protection plan becomes real here -- no merge to main without every required check green.

REQUIRED CHECKS

build and verify (backend + Angular) must both pass.

Lint and type-check gates block on Angular strict-mode errors.

A required status check list is enforced by branch protection.

At least one human review, distinct from the PR's author.

MERGE DISCIPLINE

- main is protected -- no direct pushes, ever.
- A PR description links a backlog item or ADR, per Module 48's plan.
- Failing or skipped checks block the merge button entirely.
- Force-push to main is disabled at the repository level.

Branch protection rule (concept)

```
Require status checks: build, verify, lint -- before merging
Require 1 approving review; dismiss stale reviews on new commits
```

KEY TAKEAWAY A PR that merges with a red or skipped check is exactly the gate-skipping shortcut this module's rubric is built to catch.

TRY IT YOURSELF — EXERCISE 1: DEFINE THE PR GATE POLICY

Reinforces: the capstone delivery architecture and pull-request quality gates you just saw, written as a policy that says what blocks a merge.

STEPS (notes/lab51-pr-gate-policy.md)

Step	What To Do
1 — Gate List	List the checks that must pass on every pull request before merge.
2 — Blocking vs Advisory	Mark each gate blocking or advisory, and say who can override.
3 — Fast Feedback	Name the target PR run time and what you would drop to hit it.
4 — Scope	Policy only — no workflow YAML written in this exercise.

PR gate policy

```
blocking : mvn verify, ng test, dependency scan (High+), SAST
advisory : coverage delta, image size, lint warnings
target   : PR feedback under 10 minutes; deploy never on PR
```

TRY IT NOW — Complete Exercise 1 before continuing: exercises/exercise-01-pr-gate-policy.md (workspace: examples/module-51-exercises).

3 Pull Request Quality Gates

Ensure only high-quality, secure, and tested code is merged.

Quality First. Always.

Pull Request Flow

1. **Developer Push**
   Code pushed to feature branch and PR is created.
2. **Pull Request Created**
   Changes are proposed and reviewed.
3. **GitHub Actions Triggered**
   CI pipeline runs automatically for the PR.
4. **Quality Gates Evaluation**
   All checks must pass before merge is allowed.
5. **Merge Approved**
   PR can be merged only when all quality gates pass.

Quality Gates (All Must Pass)

1. Code Build
Project builds successfully
- Maven / npm build

2. Unit & Integration Tests
All automated tests must pass
- JUnit
- Integration Tests

3. Code Quality
Code must meet quality standards
- SonarQube
- Quality Score threshold

4. Security Scans
No critical or high severity issues
- CodeQL
- Dependency Check

5. Lint & Format
Code style and format checks
- ESLint / Prettier
- Checkstyle

6. Coverage
Test coverage meets threshold
- Minimum coverage %

Gate Passed
PR is eligible for review and merge.

Branch Protection Rules

- Require pull request before merging
- Require approvals (1 or more reviewers)
- Dismiss stale pull request approvals
- Require status checks to pass
- Require branches to be up to date
- Restrict who can push to main branches
- Require signed commits

Status Checks (Examples)

- build-backend
- build-frontend
- unit-tests
- integration-tests
- codeql-analysis
- dependency-scan
- sonarqube-quality-gate
- lint-check
- test-coverage
- security-scan
- docker-build

Why Quality Gates Matter

- Prevents bugs from entering the main branch
- Improves code quality and maintainability
- Detects security vulnerabilities early
- Ensures consistent standards across the team
- Builds confidence in every deployment

Review Checklist

- Is the code easy to read and understand?
- Are edge cases handled?
- Are tests added or updated?
- Is documentation updated (if required)?

Merge Strategy

main (Protected Branch) → feature/* (Pull Request)

Automation Benefits

- Faster feedback
- Reduced manual effort
- Higher code quality
- Safer, more reliable releases

Remember

No shortcuts.
No skipped checks.
Only high-quality code makes it to production.

AUTOMATED JUNIT AND INTEGRATION TESTING

The verify stage runs the exact same test suite Modules 49 and 50 already built -- now enforced as a pipeline gate.

WHAT VERIFY RUNS

Backend: unit, MockMvc, JPA, and Kafka integration tests.

New in this module: AuthorizationTests (401/403/200).

Angular: unit and component tests from Module 50's suite.

A red test suite stops the pipeline before scan or publish.

AUTOMATION DISCIPLINE

- No test gets skipped or commented out to force a pipeline pass.
- Flaky tests get fixed, not silently retried until green.
- Test reports (Surefire, Angular) are preserved as pipeline artifacts.
- The same test suite runs locally and in CI -- no CI-only leniency.

Verify stage (concept)

```
./mvnw -B clean verify        # backend: unit + MockMvc + JPA + Kafka IT
npm ci && npm test -- --watch=false   # Angular unit + component tests
```

KEY TAKEAWAY A pipeline that skips a failing test to keep moving is functionally identical to having no test gate at all.

MODULE 51: CAPSTONE SECURITY, CI/CD AND DEPLOYMENT

Automated JUnit and Integration Testing

Ensure code quality and reliability with automated unit and integration tests as part of the CI pipeline.

KEY COMPONENTS

Automatic Test Execution
JUnit tests run automatically on every push and pull request to catch issues early.

Unit Testing with JUnit 5
Fast, isolated tests verify individual components and business logic in the service and repository layers.

Integration Testing
@SpringBootTest and test slices validate components together with real wiring and a test database.

Test Reports & Results
Generate and publish test reports for visibility and quality assurance.

Quality Gates
Fail the pipeline if tests fail, preventing broken code from progressing.

CI PIPELINE STAGE – TEST JOB

1. Checkout Code → 2. Setup JDK (Temurin 17) → 3. Cache Dependencies → 4. Run Maven Tests → 5. Publish Test Reports → 6. Quality Gate

Tests Pass
Pipeline Continues

Tests Fail
Pipeline Fails

EXAMPLE: GITHUB ACTIONS – TEST JOB

```yaml
- name: Run JUnit and Integration Tests
  run: mvn -B clean verify

- name: Publish Test Results
  uses: actions/upload-artifact@v4
  with:
    name: junit-test-results
    path: target/surefire-reports/
  if: always()

# Fails the job automatically if any test fails (default behavior)
```

BENEFITS

- Early defect detection improves code quality
- Faster feedback for developers
- Prevents regressions from reaching production
- Builds confidence in every release

Automated tests are your safety net—run them early, run them often, and keep your pipeline green.

ANGULAR BUILD AND TEST STAGE

The build stage produces one Angular production artifact, deterministically, that every later stage trusts without rebuilding.

BUILD STAGE RESPONSIBILITIES

npm ci installs from a locked, reproducible package-lock.json.

ng lint and ng build --configuration production run once.

Fail fast on a lint or compile error -- no partial artifact moves forward.

The dist/ output is tagged with version and commit SHA immediately.

AUTOMATION DISCIPLINE

- The same dist/ build is what gets tested, scanned, and containerized.
- No environment-specific rebuild later in the pipeline.
- Build and test logs are preserved as an artifact for later review.
- A flaky Angular build stage undermines trust in every stage downstream.

Angular CI stage (concept)

```
npm ci && npm run lint && npm test -- --watch=false
npm run build -- --configuration production
```

KEY TAKEAWAY Rebuilding the Angular artifact separately for test, scan, and deploy stages is exactly the provenance-destroying mistake this pipeline design prevents.

TRY IT YOURSELF — EXERCISE 2: PLAN THE ANGULAR BUILD JOB

Reinforces: the automated testing and Angular build/test stage you just covered, planned as an explicit job so the frontend cannot be silently skipped.

STEPS (notes/lab51-angular-job.md)

Step	What To Do
1 — Step Order	Write the job steps in order: checkout, setup-node, npm ci, test, build.
2 — Deterministic Install	Say why npm ci is used rather than npm install, and what it needs.
3 — Headless Tests	Name the flags that make ng test run once, headless, in CI.
4 — Build Output	Name the artifact the build produces and where it goes next.

Angular job sketch

```
setup-node 20 + cache npm   ->   npm ci        (needs package-lock.json)
ng test --watch=false --browsers=ChromeHeadless
ng build --configuration production  ->  dist/  uploaded as artifact
```

TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-angular-job-plan.md (workspace: examples/module-51-exercises).

Angular Build and Test Stage

Build the Angular application, run unit tests, and produce optimized artifacts for deployment.

KEY RESPONSIBILITIES

Install Dependencies
Use npm ci for clean, reproducible installs based on package-lock.json.

Run Unit Tests
Execute Angular unit tests with Karma and Jasmine.

Build for Production
Generate optimized production build with Angular CLI.

Publish Artifacts
Upload the dist/ folder as a build artifact for downstream stages.

Quality Gate
Fail the pipeline if tests fail or the build has errors.

ANGULAR BUILD AND TEST PIPELINE

Success — Pipeline Continues

Failure — Pipeline Fails

EXAMPLE: GITHUB ACTIONS STEP

```yaml
- name: Setup Node.js
  uses: actions/setup-node@v4
  with:
    node-version: '20'

- name: Install Dependencies
  run: npm ci

- name: Run Angular Unit Tests
  run: npm run test -- --watch=false --browsers=ChromeHeadless

- name: Build Angular App
  run: npm run build -- --configuration=production
```

BUILD OUTPUT

```
dist/
├── index.html
├── main.<hash>.js
├── polyfills.<hash>.js
├── styles.<hash>.css
├── assets/
│   └── ...
├── environments/
│   └── ...
└── favicon.ico
```

- Optimized
- Minified
- Production Ready

BENEFITS

- Catches issues early before deployment
- Faster feedback to developers
- Ensures production-ready builds
- Consistent, repeatable build process

Automated builds and tests ensure quality, reduce risk, and accelerate delivery.

SECURITY SCANNING STAGE

Scan stages run after verify and before publish -- an artifact never reaches the registry unscanned.

Scan Stage	Purpose
secret-scan	Runs first -- a leaked credential blocks everything downstream.
dep-scan	Checks resolved npm and Maven dependencies against known CVEs.
SAST	Reads Angular and Spring Boot source without running it.
DAST + image-scan	Probes a running instance and inspects the built container.

SCANNING DISCIPLINE

Reports Sanitized

Scan reports get linked from evidence docs with secrets redacted.

Time-Bound Exceptions

A critical finding needs a fix or an owned, dated exception -- never silence.

Config Lives in the Pipeline

Scan configuration lives in the workflow file, not a developer's laptop.

Re-Scan on Bump

Re-run scans after any dependency or base-image update.

KEY TAKEAWAY Reordering deploy before scans, even temporarily to 'unblock' a demo, is exactly the gate-skipping shortcut this module's rubric is built to catch.

Security Scanning Stage

Automatically scan code, dependencies, and containers to find vulnerabilities early and enforce quality gates.

WHAT WE SCAN

Source Code (SAST)
Static analysis of application source code for security issues.

Dependencies
Check open source libraries for known vulnerabilities (SCA).

Container Image
Scan container layers and installed packages for vulnerabilities.

Infrastructure as Code
Scan Terraform/Ansible manifests and configurations for misconfigurations.

Secrets and Sensitive Data
Detect hardcoded secrets, tokens, and sensitive information.

SECURITY SCANNING STAGE IN PIPELINE

1. Build Artifacts → 2. Security Scanning (SAST • SCA • Container Scan • IaC Scan • Secrets Scan) → 3. Quality Gate Evaluation → 4. Publish Artifacts → 5. Deploy to Environment

No Critical / High Issues
Pipeline Continues

Critical / High Issues Found
Pipeline Fails (Blocked)

EXAMPLE: GITHUB ACTIONS SECURITY SCANNING STEPS

```yaml
- name: Checkout Code
  uses: actions/checkout@v4
- name: Run CodeQL (SAST)
  uses: github/codeql-action/init@v3
- name: Dependency Scan (SCA)
  uses: actions/dependency-review-action@v3
- name: Container Scan
  uses: aquasecurity/trivy-action@v0.14.0
- name: Upload Scan Results
  uses: actions/upload-artifact@v4
```

SCAN RESULTS SUMMARY (EXAMPLE)

Critical	2
High	6
Medium	10
Low	6

Quality Gate: Failed
Critical or High severity vulnerabilities detected. Fix issues before proceeding.

BENEFITS

- Detect vulnerabilities early in the pipeline
- Reduce security risk and remediation cost
- Enforce security and compliance policies
- Protect applications and customer data

Shift security left – secure the pipeline, secure the product.

SAST WITH GITHUB WORKFLOW

Read the source without running it -- SAST catches patterns a human reviewer can miss at scale, across Angular and Spring Boot alike.

WHAT SAST CATCHES

Hardcoded secrets or credentials in Angular or Spring Boot source.

Injection-prone patterns (SQL, command, path).

Insecure defaults left in configuration files.

A dependency already known to carry an unsafe usage pattern.

RUNNING SAST HERE

- Runs as a required GitHub Actions job, not an optional local check.
- GitHub code scanning (CodeQL) or an equivalent SAST action, wired into the workflow.
- Findings get triaged -- fixed or given a time-bound exception.
- A SAST failure must be able to fail the job, not just annotate a PR.

Pipeline gate order (excerpt)

```
build -> verify -> secret-scan -> dep-scan -> SAST ->
image-build -> image-scan -> publish -> deploy
```

KEY TAKEAWAY A SAST gate that only annotates a PR, and never actually fails the job, is not a gate -- it is a suggestion nobody is required to read.

MODULE 51: CAPSTONE SECURITY, CI/CD AND DEPLOYMENT

SAST with GitHub Workflow

Use GitHub CodeQL to perform Static Application Security Testing (SAST) on every push and pull request to detect security vulnerabilities early.

WHAT IS SAST?

CodeQL

- Analyzes source code without executing it.
- Detects vulnerabilities, poor practices, and security hotspots.
- Supports multiple languages.
- Powered by GitHub CodeQL.

SAST WITH GITHUB WORKFLOW – PIPELINE OVERVIEW

1. Push / PR Trigger → 2. Checkout Code → 3. Initialize CodeQL → 4. Autobuild (If needed) → 5. Perform CodeQL Analysis → 6. Upload Results → 7. Show Results in Security Tab

- No Severe Issues — Pipeline Continues
- High / Critical Issues — Fail Quality Gate (Blocks Merge)

WHAT CODEQL (SAST) FINDS

- **Injection Flaws** — SQL Injection, Command Injection, XSS
- **Broken Authentication** — Weak auth logic, hardcoded secrets
- **Sensitive Data Exposure** — Hardcoded credentials, tokens, PII leaks
- **Security Misconfigurations** — Unsafe use of libraries and frameworks
- **Poor Coding Practices** — Insecure patterns and risky functions

EXAMPLE: GITHUB ACTIONS WORKFLOW (CODEQL SAST)

```yaml
name: CodeQL - SAST

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  analyze:
    name: Analyze (CodeQL)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: github/codeql-action/init@v3
        with:
          languages: java, javascript-typescript, sql
      - uses: github/codeql-action/autobuild@v3
      - uses: github/codeql-action/analyze@v3
```

CodeQL

RESULTS IN GITHUB

Code | Issues | Pull requests | Actions | Security | ...

Code scanning alerts

CodeQL — Active

Alerts by severity

Critical	2
High	5
Medium	12
Low	7
Total alerts	26

Recent alerts

- Critical — SQL query built from user input in UserRepository.java:45 — java
- High — Cross-site scripting (XSS) in product.component.ts:88 — ts
- Medium — Use of weak hash function in AuthService.java:120 — java

View all alerts

KEY BENEFITS

- Find vulnerabilities early in SDLC
- Prevent security issues from reaching production
- Improve code quality and developer awareness
- Enforce policies with quality gates
- Built-in, automated, and always up to date

Secure your code. Protect your users. Ship with confidence.

DAST INTEGRATION

Probe a running instance from the outside -- DAST finds what only shows up once the Angular UI and API are actually live on OpenShift.

WHAT DAST CATCHES

Real 401/403/200 behavior against live, deployed endpoints.

Response headers that leak information (server version, stack traces).

Actuator endpoints that are accidentally public.

CORS misconfiguration only visible from an actual Angular-origin request.

DAST VS. THE MANUAL PROBES

- Overlaps with the smoke tests covered later this module, on purpose.
- Catches runtime configuration drift SAST cannot see in source.
- Runs against a deployed instance -- staging OpenShift before production.
- Findings still require a fix or a documented, time-bound exception.

Smoke-style DAST probe (excerpt)

```
curl -fsS "$CRM_URL/actuator/health/readiness"
test "$(curl -s -o /dev/null -w '%{http_code}' "$CRM_URL/api/customers")" = "401"
```

KEY TAKEAWAY A SAST-clean codebase can still fail DAST if the deployed OpenShift configuration drifts from what the source actually intended.

MODULE 51: CAPSTONE SECURITY, CI/CD AND DEPLOYMENT

DAST Integration

Integrate Dynamic Application Security Testing (DAST) into your CI/CD pipeline to test running applications for vulnerabilities from an attacker's perspective.

WHAT IS DAST?

DAST

- Tests a running application from the outside.
- Simulates real-world attacks.
- Finds runtime vulnerabilities in web applications and APIs.
- Complements SAST (code) and SCA (dependencies).

WHAT DAST DETECTS

- **Injection Flaws** — SQL Injection, Command Injection, NoSQL Injection
- **Authentication & Session Issues** — Broken auth, session fixation, weak tokens
- **Sensitive Data Exposure** — PII leaks, directory listing, verbose errors
- **Configuration & Deployment Issues** — Missing security headers, misconfigurations
- **Business Logic Flaws** — IDOR, rate limiting issues, privilege escalation

DAST INTEGRATION IN CI/CD PIPELINE

1. Build & Test Application → 2. Deploy to Test Environment → 3. Run DAST Scan (OWASP ZAP) → 4. Evaluate Results → 5. Publish Reports → 6. Fail Pipeline on High/Critical Issues

- No High / Critical Issues — Pipeline Continues
- High / Critical Issues Found — Fail Quality Gate

EXAMPLE: GITHUB ACTIONS WORKFLOW (DAST WITH OWASP ZAP)

```yaml
name: DAST - OWASP ZAP Scan
on:
  workflow_run:
    workflows: [ "Deploy to Test" ]
    types: [ completed ]
jobs:
  dast:
    runs-on: ubuntu-latest
    steps:
    - name: Checkout Code
      uses: actions/checkout@v4
    - name: Run OWASP ZAP Baseline Scan
      uses: zaproxy/action-baseline@v0.12.0
      with:
        target: ${{ secrets.TEST_APP_URL }}
        cmd_options: -a -r zap-report.html -J zap-report.json
```

OWASP ZAP

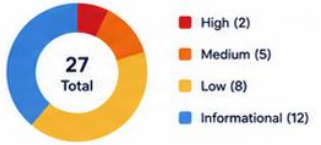
EXAMPLE DAST RESULTS (OWASP ZAP)

OWASP ZAP Baseline Report

Summary

Risk Level		
	High	2
	Medium	5
	Low	8
	Informational	12
Total Alerts	27	
Scan Duration	00:15:42	
Target	https://app.example.com	

Alerts by Risk Level



Top Alerts

High	SQL Injection	https://app.example.com/login	Active
High	Cross-Site Scripting (XSS)	https://app.example.com/search	Active
Medium	Missing Security Header	https://app.example.com/	Active

Full report: zap-report.html

KEY BENEFITS

- Find runtime vulnerabilities before attackers do
- Improve application security posture
- Automate security testing in CI/CD
- Reduce risk and protect business-critical apps
- Ensure compliance and customer trust

DAST finds what code analysis can't—vulnerabilities that exist when your application runs.

CONTAINER IMAGE BUILD

Multi-stage, non-root Dockerfiles for both the Spring Boot API and the Angular UI -- never a floating tag.

DOCKERFILE DISCIPLINE

Multi-stage build: compile/build stage, then a slim runtime stage.

The runtime stage runs as a non-root user, always.

Angular's dist/ is served from a minimal nginx or static runtime image.

No secrets, kubeconfigs, or .env files ever get baked into a layer.

IMAGE OPTIMIZATION

- A pinned base image tag, not :latest, for reproducible builds.
- Layer caching orders least-to-most-frequently-changed instructions.
- .dockerignore excludes node_modules, target/, and test fixtures.
- Each image gets tagged with version and commit SHA at build time.

Multi-stage build (concept)

```
FROM eclipse-temurin:21-jdk AS build ... FROM gcr.io/distroless/java21-debian12
USER nonroot   # never run the container as root
```

KEY TAKEAWAY An image built as root, from an unpinned base, is a graded security and provenance failure -- multi-stage and non-root are the minimum bar.

Container Image Build

Build a secure, optimized container image from application artifacts to ensure consistency across environments and enable reliable deployments.

WHY BUILD CONTAINER IMAGES?

Consistency
Package the application and all dependencies in a standardized image.

Portability
Run the container image anywhere – dev, test, staging, or production.

Isolation
Ensure application isolation and reduce "it works on my machine" issues.

Efficiency
Leverage layered images for faster builds and smaller sizes.

Security
Control base images, include only what's needed, and scan for vulnerabilities.

CONTAINER IMAGE BUILD IN CI/CD PIPELINE

Build Successful
Pipeline Continues

Build Failed
Pipeline Fails

EXAMPLE: GITHUB ACTIONS – BUILD IMAGE

```yaml
- name: Build Docker Image
  uses: docker/build-push-action@v5
  with:
    context: .
    file: ./Dockerfile
    tags: |
      myorg/employee-api:${{ github.sha }}
      myorg/employee-api:latest
    push: false
  platforms: linux/amd64

- name: Image Build Summary
  run: echo "Image built successfully"
  shell: bash
```

EXAMPLE: DOCKERFILE (SPRING BOOT APP)

```dockerfile
FROM eclipse-temurin:17-jdk-alpine AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN ./mvnw clean package -DskipTests

FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar

EXPOSE 8080
ENTRYPOINT ["java","-jar", "app.jar"]
```

KEY BENEFITS

- Reproducible builds across environments
- Faster deployments and rollbacks
- Smaller attack surface and better security
- Easy to version and trace
- Supports scalable, modern deployments

Container images package your application consistently and securely—build once, run anywhere.

CONTAINER SECURITY SCAN

Dependency, secret, and image scans each surface a different exposure -- run all three against both containers, not just one.

THREE SCAN TYPES

Dependency scan: known-vulnerable npm and Maven libraries by CVE.

Secret scan: credentials or tokens committed to Git history.

Image scan: OS packages and layers inside both built containers.

Each catches a class of risk the other two cannot see.

ASSESSMENT DISCIPLINE

- Critical findings block the pipeline unless explicitly exceptioned.
- Exceptions are time-bounded, with an owner and expected fix date.
- Ignoring a critical CVE silently is a named failure -- fix or document it.
- Scan reports are sanitized before they are linked in evidence docs.

Digest inspection (concept)

```
docker image inspect "$REGISTRY/crm-api:${VERSION}-${GIT_SHA}" \
  --format='{{index .RepoDigests 0}}'
```

KEY TAKEAWAY Ignoring a critical CVE in either the Angular or the Spring Boot image without an honest, owned risk entry is explicitly named as a failure this module does not accept.

TRY IT YOURSELF — EXERCISE 3: PLAN SCANNING AND IMAGE IDENTITY

Reinforces: the SAST, DAST, container build, and container scanning stages you just walked, turned into thresholds and a digest-based image identity.

STEPS (notes/lab51-scan-image-plan.md)

Step	What To Do
1 — Scanner Table	List each scanner, what it inspects, and the severity that fails the build.
2 — Residual Risk	Say how an accepted finding is recorded, with an owner and an expiry.
3 — Image Identity	State that the image is referenced by digest, never by a floating tag.
4 — Scan Before Push	Say where the image scan runs relative to pushing to the registry.

Scan gates and identity

```
dependency scan -> libraries   fail at High
SAST            -> our source   fail at High
image scan      -> OS + layers  fail at Critical, scan BEFORE push
```

TRY IT NOW — Complete Exercise 3 before continuing: exercises/exercise-03-scan-and-image-plan.md (workspace: examples/module-51-exercises).

MODULE 51: CAPSTONE SECURITY, CI/CD AND DEPLOYMENT

Container Security Scan

Scan container images for known vulnerabilities, misconfigurations, and secrets to ensure secure deployments and reduce risk in production.

WHY SCAN CONTAINERS?

Detect Vulnerabilities
Find OS packages and application libraries with known CVEs.

Reduce Risk
Catch issues early before images are deployed to environments.

Ensure Compliance
Meet security and regulatory requirements with automated scans.

Improve Trust
Validate that only secure, approved images move through the pipeline.

Fail Fast
Block high or critical issues and enforce quality gates.

CONTAINER SECURITY SCAN IN CI/CD PIPELINE

1. Build Container Image → 2. Push to Registry → 3. Security Scan (Trivy / Grype) → 4. Evaluate Results → 5. Quality Gate Decision → 6. Continue or Block Deployment

- No High / Critical Issues — Pipeline Continues
- High / Critical Issues Found — Pipeline Fails (Blocked)

EXAMPLE: GITHUB ACTIONS – TRIVY SCAN STEP

```yaml
- name: Scan Docker Image with Trivy
  uses: aquasecurity/trivy-action@v0.20.0
  with:
    image-ref: ${{ steps.meta.outputs.tags }}
    format: 'table'
    exit-code: '1'        # fail on HIGH/CRITICAL
    ignore-unfixed: true
    vuln-type: 'os,library'
    severity: 'CRITICAL,HIGH'
    scanners: 'vuln,config,secret'
    cache: true
    timeout: '10m'
    output: trivy-results.txt
    output-format: 'sarif'
```

trivy

EXAMPLE: SCAN RESULTS (TRIVY TABLE)

Total: 128 | CRITICAL: 2 | HIGH: 9 | MEDIUM: 34 | LOW: 83

TYPE	SEVERITY	VULNERABILITY ID	PACKAGE	INSTALLED VERSION	FIXED VERSION
os	CRITICAL	CVE-2024-3094	glibc	2.35-r0	2.35-r2
os	HIGH	CVE-2024-21626	bash	5.1.16-r0	5.1.16-r1
library	HIGH	CVE-2024-25710	spring-core	6.0.8	6.0.13
library	MEDIUM	CVE-2023-34034	jackson-databind	2.15.2	2.15.3
...	...	...	...	...	...

Detailed report: trivy-results.txt

KEY BENEFITS

- Find vulnerabilities before deployment
- Reduce attack surface of container images
- Automate security quality gates
- Protect production environments
- Build secure, trusted containers

Scan early. Fix fast. Deploy secure.

ARTIFACT PROMOTION

One built, scanned, digest-identified artifact is promoted through every environment -- never rebuilt per stop.

Digest, Not Tag

The promoted identity is the image digest -- a tag alone is not proof of what actually ships.

Registry Push Once

The image is pushed to the registry exactly once, after every gate has passed.

Promotion Is Gated

Moving from staging to production requires GitHub Environments approval, not a manual push.

Provenance Recorded

Tag, digest, pipeline run id, and git SHA are all recorded together, every promotion.

KEY TAKEAWAY Rebuilding the image at any promotion step reintroduces exactly the provenance gap this module's single-build pipeline exists to prevent.

MODULE 51: CAPSTONE SECURITY, CI/CD AND DEPLOYMENT

Artifact Promotion

Promote the same, verified artifact through environments without rebuilding to ensure consistency, traceability, and auditability.

ARTIFACT PROMOTION PIPELINE

1. Build & Test (Create Artifact) → 2. Store Artifact (Registry/Storage) → 3. Promote to DEV (Approved Artifact) → 4. Promote to QA (Same Artifact) → 5. Promote to UAT (Same Artifact) → 6. Promote to PROD (Same Artifact)

BUILD ONCE, PROMOTE MANY TIMES
The exact same artifact moves through environments. No rebuilds.

INTEGRITY & TRUST
Verified once, trusted everywhere.

WHY PROMOTE ARTIFACTS?

Consistency
The same artifact is used in every environment, reducing "it works on my machine" issues.

Traceability
Full traceability from build → deploy → run in production.

Security
Scan and verify once, then promote trusted artifacts.

Efficiency
Faster deployments by eliminating rebuilds in downstream environments.

Auditability
Clear audit trail of who promoted what, when, and to which environment.

PROMOTION STRATEGY

Build & Verify
Build the application and run all tests, scans, and quality gates.

↓

Publish Artifact
Push the artifact (container image / package) to a registry or artifact repository.

↓

Promote to DEV
Deploy the approved artifact to DEV environment.

↓

Promote to QA
After validation, promote the same artifact to QA.

↓

Promote to UAT
Promote the same artifact to UAT for business validation.

↓

Promote to PROD
After final approval, promote the same artifact to production.

EXAMPLE: GITHUB ACTIONS – PROMOTE ARTIFACT

```yaml
- name: Promote Artifact to QA
  uses: actions/download-artifact@v4
  with:
    name: app-image
    path: ./artifact

- name: Deploy to QA
  run: |
    IMAGE=${{ env.REGISTRY }}/app:${{ github.sha }}
    echo "Deploying $IMAGE to QA"
    kubectl --context qa set image deployment/app \
      app=$IMAGE --record

- name: Tag as QA Approved
  run: |
    echo "QA_APPROVED" > ./artifact/status.txt
```

ARTIFACT DETAILS (EXAMPLE)

Artifact Name	:	app-image
Version	:	1.2.3+build.45
Commit	:	a1b2c3d
Built By	:	github-actions
Built At	:	2025-05-17 10:15 UTC
Type	:	Docker Image
Registry	:	ghcr.io/org/app
Digest	:	sha256:10f9c...8ab2
Status	:	QA Approved

PROMOTION HISTORY

Environment	Promoted By	When	Status
DEV	alice	2025-05-17 11:02 UTC	SUCCESS
QA	bob	2025-05-17 14:20 UTC	SUCCESS
UAT	carol	2025-05-17 16:45 UTC	SUCCESS
PROD	dave	2025-05-18 09:10 UTC	PENDING

KEY BENEFITS

- Reduce risk with immutable artifacts
- Ensure compliance and approvals
- Accelerate delivery across environments
- Improve collaboration and visibility
- Consistent, proven artifacts in production

Promote with confidence. Deliver with consistency.

TERRAFORM INFRASTRUCTURE STAGE

Module 48's Terraform plan becomes a real, gated pipeline stage here -- provisioning the OpenShift project before any manifest applies.

TERRAFORM STAGE STEPS

terraform plan runs as a GitHub Actions job, its output posted to the PR.

terraform apply runs only after a human reviews the plan output.

State is stored remotely and locked -- never a local .tfstate file.

The OpenShift Project, quotas, and network policy are the managed resources.

WHY IT MATTERS FOR RELEASE

- A project or quota drift can silently break the OpenShift deploy stage.
- IaC changes go through the same PR review as application code.
- Reproducible infrastructure is what makes the evidence trail trustworthy.
- This stage runs before the deploy stage, never applied ad hoc mid-incident.

Plan-before-apply, as a pipeline stage

```
terraform plan -out=tfplan   # posted as a PR comment for review
terraform apply tfplan       # runs only after approval, on merge to main
```

KEY TAKEAWAY An unreviewed terraform apply run directly against the shared OpenShift cluster is exactly the undocumented change this pipeline stage prevents.

Terraform Infrastructure Stage

Infrastructure as Code with Terraform

MODULE 51
Capstone Security, CI/CD and Deployment

Provision, manage, and version infrastructure with Terraform

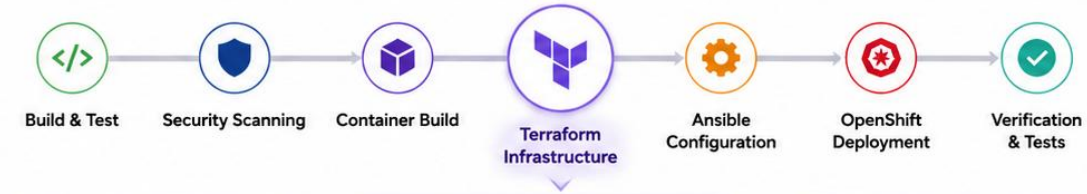
The Terraform Infrastructure Stage provisions and manages all infrastructure required for the application using Infrastructure as Code (IaC). It ensures consistency, repeatability, and traceability across environments.

Key Objectives

- Provision cloud infrastructure (OpenShift, VPC, Networking)
- Manage infrastructure as code in version control
- Enforce environment consistency
- Enable automated, repeatable deployments
- Support multi-environment promotion

Benefits

- Consistent environments
- Faster provisioning
- Version-controlled infrastructure
- Reduced manual errors



Terraform Workflow

terraform init
Initialize providers and modules

terraform plan
Preview infrastructure changes

terraform apply
Provision or update infrastructure

terraform state
Maintain infrastructure state

Terraform Configuration (Example)

```
provider "aws" {
  region = var.aws_region
}

resource "aws_vpc" "app_vpc" {
  cidr_block = "10.0.0.0/16"
  tags = {
    Name = "capstone-vpc"
  }
}

resource "aws_subnet" "app_subnet" {
  vpc_id            = aws_vpc.app_vpc.id
  cidr_block        = "10.0.1.0/24"
  availability_zone = "us-east-1a"
  tags = {
    Name = "capstone-subnet"
  }
}
```

Provisioned Infrastructure

- Virtual Private Cloud Networking
- Subnets & Route Tables
- Security Groups & Network Policies
- Managed Services (RDS, S3, etc.)
- OpenShift Cluster Resources

Outcome: Infrastructure is provisioned automatically, securely, and consistently across environments.

Result: A reliable and repeatable foundation for application deployment.

ANSIBLE CONFIGURATION STAGE

Configuration management runs as its own gated stage, keeping OpenShift nodes and shared tooling consistent before the deploy stage begins.

ANSIBLE STAGE STEPS

A playbook run configures shared tooling ahead of the OpenShift deploy.

Idempotency is verified -- a second run reports zero changes.

Inventory separates staging and production -- no shared secrets.

Ansible Vault (or an equivalent) protects every sensitive variable.

WHY IT MATTERS FOR RELEASE

- A manually patched node is undocumented and unreproducible.
- Consistent baseline configuration reduces surprising deploy failures.
- Playbook changes are reviewed, matching the Terraform stage's discipline.
- This stage runs before deploy -- never a manual fix mid-incident.

Idempotency check, as a pipeline gate

```
ansible-playbook site.yml --check --diff
# CI fails the job if an unexpected diff is reported
```

KEY TAKEAWAY A node fixed manually during an incident, with no playbook update to match, drifts from the documented baseline the moment anyone forgets exactly what they did.

Ansible Configuration Stage

Automate Configuration, Setup, and Orchestration

MODULE 51
Capstone Security, CI/CD and Deployment

Configure applications and environments consistently with Ansible

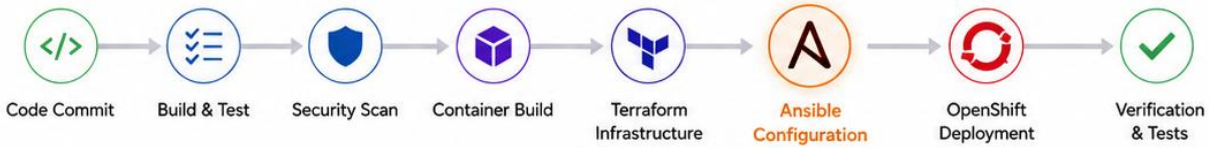
The Ansible Configuration Stage automates the setup and configuration of servers, applications, middleware, and system parameters. It ensures consistency, idempotency, and repeatability across all environments.

Key Objectives

- Automate server and application configuration
- Ensure consistent environments across all nodes
- Configure middleware, services, and system settings
- Manage secrets and configuration securely
- Enable idempotent, repeatable deployments
- Reduce manual errors and configuration drift

Benefits

- Consistent and reliable environments
- Fast and automated configuration
- Idempotent and repeatable execution
- Reduced manual effort
- Easy maintenance and scaling
- Auditability and visibility



Ansible Workflow

Inventory
Define target hosts and groups

↓

Playbook
YAML-based automation and configuration

↓

Roles
Reusable components for configuration

↓

Playbook Execution
Run ansible-playbook against inventory

↓

Configuration Applied
Validate and verify configuration state

Example: Playbook (site.yml)

```yaml
---
- name: Configure Application Servers
  hosts: app_servers
  become: yes
  vars:
    app_user: appuser
    app_port: 8080
  roles:
    - common
    - java
    - application
  tasks:
   - name: Ensure application directory
     file:
       path: /opt/myapp
       state: directory
       owner: "{{ app_user }}"
       group: "{{ app_user }}"
       mode: '0755'
   - name: Deploy configuration file
     template:
       src: app.conf.j2
       dest: /opt/myapp/app.conf
     notify: Restart Application
  handlers:
   - name: Restart Application
     service:
       name: myapp
       state: restarted
```

Configuration Outcomes

OS & System Settings
Users, packages, updates, sysctl, firewall

Application Configuration
App configs, environment variables, templates

Middleware Setup
Web servers, message brokers, databases, caching

Security Hardening
SSH, permissions, policies, security baselines

Service Management
Start/stop/restart services, health checks

Outcome: Environments are configured automatically, securely, and consistently using Ansible automation.

Result: Reliable application environments with reduced manual errors and faster, repeatable deployments.

Ansible ensures configuration as code for every environment.

OPENSHIFT DEPLOYMENT STAGE

This is the real production deployment step -- every rollout in this module targets OpenShift directly, with no comparison cluster involved.

Action	Command / Object	Waits For
Apply manifests	oc apply -f k8s/ (Deployment, Service, Route, ConfigMap/Secret)	Resources accepted by the API server.
Wait for rollout	oc rollout status deployment/crm-api --timeout=180s	New, ready pods before old ones terminate.
Confirm Route	oc get route crm-ui -o jsonpath='{.spec.host}'	A resolvable, TLS-terminated external host.
Verify probes	readiness/liveness against /actuator/health/*	Traffic only reaches genuinely ready pods.

DEPLOY SEQUENCE

1 **Apply**

oc apply pinned to the published digest, never a floating tag.

2 **Roll Out**

oc rollout status waits for readiness before declaring done.

3 **Expose**

The Route serves the Angular UI and API over HTTPS.

4 **Verify**

Probes and logs confirm the deployment is genuinely healthy.

KEY TAKEAWAY A deploy that declares success right after oc apply, without waiting for rollout status, ships broken pods behind a green checkmark -- on the real target, not a rehearsal cluster.

TRY IT YOURSELF — EXERCISE 4: PLAN THE DEPLOYMENT STAGES

Reinforces: the artifact promotion, Terraform, Ansible, and OpenShift deployment stages you just saw, sequenced with the Lab 48 infrastructure plan.

STEPS (notes/lab51-deploy-stage-plan.md)

Step	What To Do
1 — Stage Order	Write the stages in order: promote, terraform, ansible, deploy, verify.
2 — Ownership	Say what Terraform owns and what Ansible owns, matching your Lab 48 plan.
3 — Same Artifact	State that the digest promoted is the digest deployed, unchanged.
4 — Failure Stop	Say which stage failing stops the pipeline before anything reaches users.

Deployment stage order

```
promote(digest) -> terraform apply -> ansible configure
  -> oc deploy (same digest) -> verify readiness
any stage fails -> stop; nothing partially applied is left running
```

TRY IT NOW — Complete Exercise 4 before continuing: exercises/exercise-04-deploy-stage-plan.md (workspace: examples/module-51-exercises).

OpenShift Deployment Stage

OPENSHIFT

Deploy, scale, and operate applications on OpenShift

MODULE 51
Capstone Security, CI/CD and Deployment

Deploy containers and run applications at scale on OpenShift

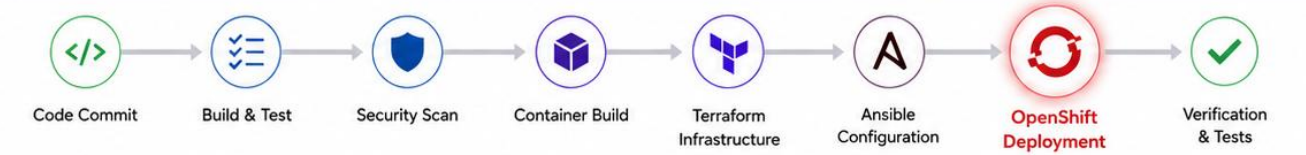
The OpenShift Deployment Stage deploys containerized applications to OpenShift clusters using Kubernetes capabilities. It ensures high availability, scalability, security, and self-healing operations in enterprise environments.

Key Objectives

- Deploy container images to OpenShift clusters
- Manage Kubernetes resources with manifests
- Enable high availability and auto-scaling
- Secure applications and network traffic
- Monitor health, logs, and performance
- Ensure zero-downtime deployments

Benefits

- Resilient and highly available apps
- Built-in security and compliance
- Automated rollouts and rollbacks
- Self-healing and auto-scaling
- Consistent, reliable deployments
- Operational visibility and control



Deployment Workflow on OpenShift

Image Push
Push container image to internal registry

Apply Manifests
Apply Kubernetes manifests (YAML) to cluster

Rolling Deployment
OpenShift performs rolling update with health checks

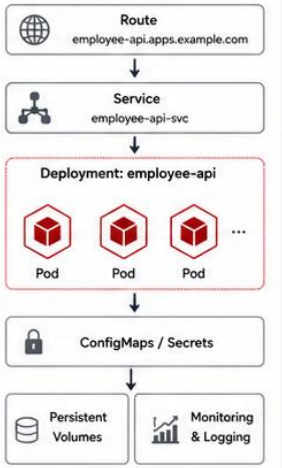
Service Exposure
Expose applications using Services and Routes

Monitoring & Health
Monitor pods, logs, metrics, and events

Example: Deployment Manifest (deployment.yaml)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: employee-api
  labels:
    app: employee-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: employee-api
  template:
    metadata:
      labels:
        app: employee-api
    spec:
      containers:
        - name: employee-api
          image: registry.internal/employee-api:1.0.0
          ports:
            - containerPort: 8080
          eny:
            - name: SPRING_PROFILES_ACTIVE
              value: "prod"
          resources:
            requests:
              cpu: "250m"
              memory: "256Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
```

OpenShift Cluster



OpenShift Capabilities

Security
RBAC, SCC, Network Policies, TLS encryption

High Availability
Multi-replica deployments and self-healing

Auto Scaling
Horizontal Pod Autoscaler and resource-based scaling

Routes
Edge/Ingress for secure external access

Rollbacks
Instant rollback to previous revisions

Observability
Metrics, logs, alerts, dashboards

Outcome: Applications are securely deployed, highly available, and scalable on OpenShift.

Result: Reliable, resilient, and observable applications running in production with zero or minimal downtime.

OpenShift provides the enterprise-grade platform to run modern applications with confidence.

Built on Kubernetes.
Optimized for Enterprise.

GITHUB ENVIRONMENTS

Staging and production are configured as separate GitHub Environments, each with its own protection rules and secrets.

Protection Rules

Required reviewers, wait timers, and branch restrictions gate every deploy job.

Required Reviewers

A named reviewer must approve before a job targeting production runs -- production requires two or more approvals, staging needs one.

Wait Timer

An optional cooling-off period, e.g. a five-minute wait timer, before a production deploy job is allowed to start.

Environment Secrets

Staging and production hold separate credentials -- never shared across environments.

KEY TAKEAWAY A deploy job with no environment protection rule is exactly one accidental merge away from an unreviewed production release.

GitHub Environments

Manage deployments across environments with protection and control

MODULE 51
Capstone Security, CI/CD and Deployment

What are GitHub Environments?

GitHub Environments allow you to model and manage deployment targets such as Dev, QA, and Production. They provide protection rules, secrets, and visibility to ensure safe and controlled deployments.

Key Benefits

- Define deployment targets and stages
- Apply protection rules and approvals
- Use environment-specific secrets and variables
- Track deployment history and status
- Increase visibility, security, and compliance

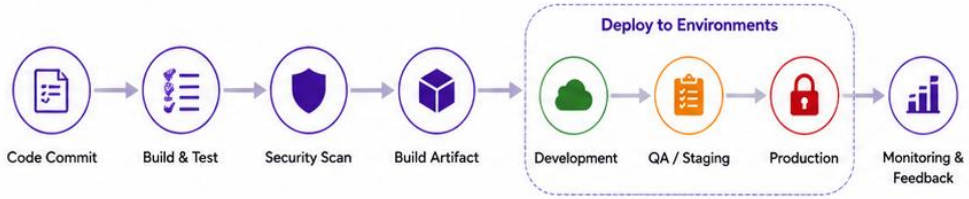
Typical Environment Stages

Development
Active development and integration

QA / Staging
Testing, validation, and UAT

Production
Live environment for end users

CI/CD Pipeline with GitHub Environments



Example: Environment Configuration

Environment	Purpose	Protection Rules	Secrets & Variables	Deployment Target
Development	Feature testing and integration	• No approvals required • Allow force pushes	• DEV_API_URL • DEV_DB_URL	dev-cluster
QA / Staging	System testing and user acceptance	• Required reviewers (1 approval) • Restrict who can deploy	• QA_API_URL • QA_DB_URL	qa-cluster
Production	Live environment for customers	• Required reviewers (2+ approvals) • Prevent self-approval • Wait timer (e.g., 5 min)	• PROD_API_URL • PROD_DB_URL	prod-cluster

Environment Protection Rules

Required Reviewers
Require approval from specified team members.

Wait Timers
Add a delay before deployment continues.

Deployment Branch Policies
Restrict which branches can deploy.

Prevent Self-Review
Disallow users from approving their own deployments.

Custom Rules
Combine rules to match enterprise compliance needs.

Goal: Deliver applications to the right environment at the right time with the right controls.

Outcome: Safer deployments, reduced risk, and full visibility across the delivery pipeline.

Tip: Use GitHub Environments with protection rules and environment secrets for enterprise-grade CD.

PRODUCTION APPROVAL GATE

A release is approved on evidence, not on confidence -- the same four artifacts get checked every time before production.

Digest Identity

Tag, digest, pipeline run id, and git SHA all recorded together.

Smoke Evidence

Both 401 anonymous and 200 authenticated proven against the deployed digest.

Scan Disposition

Every critical finding fixed or exceptioned, with an owner and expiry.

Rollback Rehearsed

A previous digest redeploy verified on OpenShift, with health confirmed after.

KEY TAKEAWAY A release approved on 'it looked fine when I checked' instead of these four artifacts is exactly the gap this module's evidence discipline is designed to expose.

Production Approval Gate

Human-in-the-loop control before production deployment

MODULE 51
Capstone Security, CI/CD and Deployment

Why an Approval Gate?

The Production Approval Gate ensures that only vetted, tested, and secure changes are deployed to production. It adds a mandatory human approval step to reduce risk and enforce compliance.

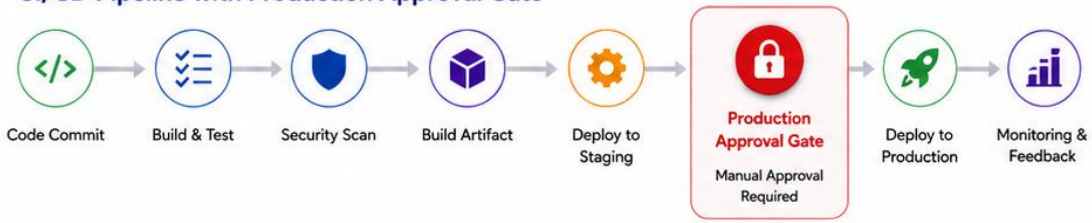
Key Benefits

- Prevents unreviewed changes from reaching production
- Ensures compliance and policy enforcement
- Provides a final checkpoint for quality and security
- Enables clear accountability and audit trail
- Reduces the risk of production incidents

Approval Gate Controls

- **Required Reviewers** — Specific users or teams must approve.
- **Wait Timer (Optional)** — Add delay before approval can be granted.
- **Status Checks** — All required checks must pass.
- **Environment Protection Rules** — Restrict who can approve and deploy.

CI/CD Pipeline with Production Approval Gate



How the Approval Gate Works

1. **Pipeline Pauses** — Pipeline waits at the approval gate after staging deployment.
2. **Notifications Sent** — Reviewers are notified via GitHub, email, or chat (Slack/Teams).
3. **Code & Release Review** — Reviewers validate changes, quality, security, and business impact.
4. **Approval / Rejection** — Approve to continue or reject to stop deployment.
5. **Pipeline Resumes** — Once approved, pipeline continues to deploy to production.

Example: GitHub Actions Environment Protection Rule

```yaml
deploy-production:
  runs-on: ubuntu-latest
  environment:
    name: production
    url: https://app.example.com
  steps:
    - name: Deploy to Production
      run: ./deploy.sh --env production
```

Environment: production

- Required reviewers: Release Managers
- Wait timer: 5 minutes
- Required status checks:
  - All tests passed
  - Security scan passed
  - Artifact available
- Deployment branches: main only

Typical Approval Criteria

- All automated tests passed
- Security scans passed
- Code review completed
- Release notes validated
- Business / Change validation
- Rollback plan confirmed

Audit & Visibility

- Approval history is recorded
- Who approved / rejected
- When the approval occurred
- What was deployed
- Full traceability in GitHub

Goal: Protect production by ensuring only approved, validated changes are deployed.

Who Approves: Designated release managers, admins, or on-call approvers.

Where Configured: GitHub → Settings → Environments → production → Protection rules

Outcome: Safer releases, reduced risk, and stronger compliance.

SECRETS AND OIDC

GitHub Actions secrets hold what must stay secret; OIDC federation removes the need for a long-lived OpenShift credential entirely.

GITHUB ACTIONS SECRETS

Registry credentials and any static API keys live in Actions secrets.

Environment-scoped secrets never cross the staging/production boundary.

Secrets are referenced by name in workflow YAML -- never inlined.

A secret value never appears in a workflow log, redacted automatically.

OIDC FEDERATION

- GitHub Actions requests a short-lived token via OpenID Connect.
- OpenShift/cloud trusts the GitHub OIDC issuer -- no stored long-lived credential.
- A token is scoped to one workflow run and expires automatically.
- Eliminates a static, leakable OpenShift service-account credential in Secrets.

OIDC vs. static secret (concept)

```
Static: OPENSHIFT_TOKEN stored forever, valid until manually rotated.
OIDC: a fresh, scoped token minted per run, valid only for that run.
```

KEY TAKEAWAY A long-lived deployment credential sitting in Actions secrets is a standing risk -- OIDC federation is the stronger default wherever it's available.

TRY IT YOURSELF — EXERCISE 5: PLAN SECRETS AND APPROVALS

Reinforces: GitHub Environments, the production approval gate, and OIDC secret handling you just covered — planned as names and owners, never values.

STEPS (notes/lab51-secrets-approvals.md)

Step	What To Do
1 — Environment Map	List the GitHub Environments and which branch or tag reaches each.
2 — Approval Gate	Name who approves production and what evidence they see first.
3 — OIDC over Secrets	Say why short-lived OIDC tokens beat long-lived stored credentials.
4 — Names Only	Confirm your notes contain secret NAMES only — never a value.

Environments and identity

```
staging  <- main pushes,  no approval needed
production <- v* tags,    required reviewer + wait timer
auth: OIDC federation -> short-lived token; no stored cloud keys
```

TRY IT NOW — Complete Exercise 5 before continuing: exercises/exercise-05-secrets-and-approvals.md (workspace: examples/module-51-exercises).

Secrets and OIDC

Secure authentication and secret management for CI/CD

MODULE 51
Capstone Security, CI/CD and Deployment

GitHub Secrets

Store sensitive information securely and use them in your workflows.

What are Secrets?

- Encrypted at rest and in transit
- Only accessible to authorized workflows
- Not exposed in logs or output
- Scoped to repository, environment, or organization

Types of Secrets

Repository Secrets
Available to all workflows in the repository

Environment Secrets
Scoped to an environment (e.g., staging, production)

Organization Secrets
Shared across repositories in an organization

Dependabot Secrets
Secure dependency updates

Best Practices

- Use least privilege
- Rotate secrets regularly
- Never hardcode secrets in code
- Limit secret access using environments and rules

Using Secrets in GitHub Actions


```yaml
- name: Deploy to Production
  run: ./deploy.sh
  env:
    DB_HOST: ${{ secrets.PROD_DB_HOST }}
    DB_USER: ${{ secrets.PROD_DB_USER }}
    DB_PASSWORD: ${{ secrets.PROD_DB_PASSWORD }}
    API_TOKEN: ${{ secrets.PROD_API_TOKEN }}
```

Secrets vs OIDC

Feature	GitHub Secrets	OIDC
Credential Type	Static (stored secret)	Dynamic (short-lived token)
Storage	Stored in GitHub	Not stored
Rotation	Manual / Periodic	Automatic
Risk	Higher (if leaked)	Lower (no static secret)
Use Case	API keys, tokens, passwords	Cloud authentication, deployments

OpenID Connect (OIDC)

Use OIDC to securely authenticate GitHub Actions with cloud providers without long-lived credentials.

Why Use OIDC?

- Eliminates static cloud credentials
- Short-lived, automatically rotated tokens
- Stronger security and compliance
- Built-in with GitHub Actions

OIDC Flow

No long-lived keys are stored in GitHub.
Access is granted based on identity and claims in the token.

Example: AWS OIDC Trust Policy

```json
{
  "Effect": "Allow",
  "Principal": {
    "Federated": "arn:aws:iam::123456789012:oidc-provider/token.actions.githubusercontent.com"
  },
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": {
      "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
      "token.actions.githubusercontent.com:sub": "repo:org/repo:ref:refs/heads/main"
    }
  }
}
```

Goal: Protect credentials and authenticate securely in CI/CD pipelines.

Secrets: Store sensitive data securely and limit access.

OIDC: Eliminate static credentials with short-lived tokens.

Outcome: Stronger security, reduced risk, and compliance-ready deployments.

DEPLOYMENT VERIFICATION

Before trusting a deploy, prove the OpenShift context, project, and access are actually what the team expects.

VERIFICATION CHECKS

oc whoami and oc project confirm the intended OpenShift context.

Registry credentials and image pull access verified ahead of deploy.

oc version confirms client/server tooling compatibility.

Readiness/liveness probe status watched over time, not just at deploy.

VERIFICATION DISCIPLINE

- Never commit a kubeconfig or an oc login token to Git, ever.
- The target Project name lives in docs -- never assumed from memory.
- A wrong-project deploy is a named, avoidable incident, not bad luck.
- Application logs stay searchable by correlation id, tokens redacted.

Pre-flight verification (concept)

```
oc whoami   &&   oc project
oc get pods -l app=crm-api   # confirm the expected pods and their status
```

KEY TAKEAWAY Registry or authentication issues at this verification step often burn real time unverified -- checking pull secrets and context early is time well spent.

Deployment Verification

Ensure the deployed application is healthy, functional, and ready for users

MODULE 51
Capstone Security, CI/CD and Deployment

Why Deployment Verification?

Deployment is not complete until the application is verified. Automated and manual checks confirm that the system is healthy, secure, and functioning as expected in the target environment.

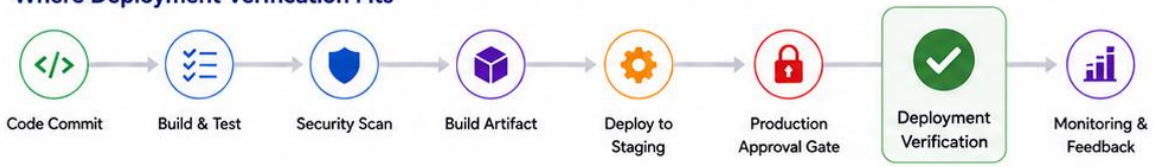
Key Objectives

- Verify successful deployment to the target environment
- Validate application health and availability
- Confirm critical business functionality
- Ensure security controls are active
- Detect issues early and prevent user impact
- Provide confidence for release completion

Verification Types

- **Automated Checks** — Scripts, APIs, and synthetic tests
- **Functional Checks** — Validate key user workflows
- **Security Checks** — Validate security headers, certificates, and access
- **Observability Checks** — Logs, metrics, alerts, and dashboards

Where Deployment Verification Fits



Deployment Verification Workflow

1. **Trigger Verification** — Starts automatically after deployment to production or on manual trigger.
2. **Run Automated Checks** — Execute health checks, API tests, smoke tests, security scans, and infrastructure checks.
3. **Validate Results** — Evaluate results and compare against expected outcomes and thresholds.
4. **Handle Failures** — Stop the pipeline, notify teams, and trigger rollback if necessary.
5. **Approval / Release** — Verification passed. Deployment is marked successful and visible to users.

Example: Automated Verification Steps

```
curl -f https://app.example.com/health                          // Application health endpoint
curl -f https://app.example.com/api/health                      // API health check
npm run e2e:smoke                                               // End-to-end smoke tests
k6 run smoke-test.js                                            // Load / performance smoke test
testssl.sh https://app.example.com                              // TLS / certificate validation
kubectl get pods -n prod                                        // Verify all pods are running
kubectl get svc -n prod                                         // Verify services are available
curl -I https://app.example.com | grep -i "strict-transport-security"  // Verify security headers

All checks passed successfully!
```

What to Verify

- **Application Availability** — Site loads, endpoints respond
- **Core Functionality** — Critical user journeys work
- **Security Posture** — TLS, headers, auth, access
- **Infrastructure Health** — Pods, services, ingress, DB
- **Observability** — Metrics flowing, no critical alerts
- **Configuration Validation** — Correct versions and settings

Best Practices

- Automate as many checks as possible
- Keep checks fast and reliable
- Validate business-critical functionality
- Fail fast and rollback when needed
- Monitor continuously after release
- Maintain and version verification scripts

Goal: Confirm the deployed system is healthy, secure, and ready for users.

Result: Successful deployments, higher quality, and faster feedback.

Who Benefits: Developers, QA, Operations, Security, and Business Stakeholders.

Outcome: Reliable releases, reduced risk, and improved user experience.

SMOKE TESTING

Both halves of the smoke test matter equally -- the authenticated success path and the anonymous denial path, on OpenShift.

SMOKE TEST SEQUENCE

Health check: actuator/health/readiness returns healthy.

Anonymous /api/customers returns exactly 401.

Authenticated read with a valid token returns 200.

Correlation id release-smoke-${BUILD} traceable in logs.

SMOKE DISCIPLINE

- Skipping the unauthorized check is a named failure-handling deduction.
- Smoke runs against the pinned digest on OpenShift, never a floating tag.
- A flaky smoke result gets investigated, not silently retried until green.
- Smoke output gets saved as sanitized evidence, tokens redacted.

Smoke sequence (concept)

```
curl -fsS "$CRM_URL/actuator/health/readiness"; test "$(curl -s -o /dev/null \
  -w '%{http_code}' "$CRM_URL/api/customers")" = "401"   # then retry authenticated -> 200
```

KEY TAKEAWAY Smoke that only proves the happy path passed is exactly half the evidence this module actually requires.

Smoke Testing

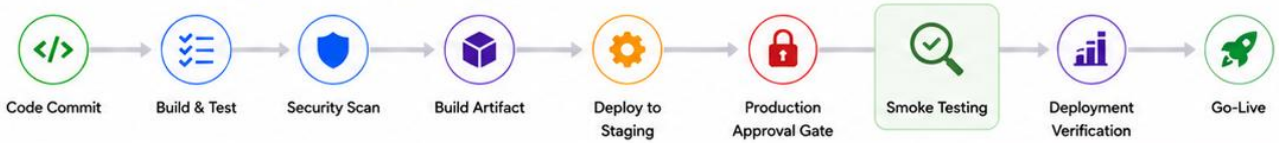
Quickly validate critical functionality after deployment

MODULE 51
Capstone Security, CI/CD and Deployment

What is Smoke Testing?

Smoke testing is a quick, high-level test to verify that the deployed application's most critical functions are working. It helps catch major issues early before broader testing or user access.

Where Smoke Testing Fits in the CI/CD Pipeline



Key Objectives

- Verify that the deployed build is stable
- Ensure critical user journeys work
- Detect show-stopper issues early
- Confirm integrations and core services are up
- Provide go/no-go signal for broader testing

Characteristics

- **Quick** — Run a small set of essential tests
- **High-Level** — Focus on major features and flows
- **Automated** — Implemented as automated scripts
- **Pass/Fail** — Minimal reporting, clear outcome

Typical Smoke Test Scope

Application Availability	Authentication	Core Services	Key User Journey	Integrations	Security
App loads successfully	Login works with valid credentials	Critical APIs and services respond successfully	Happy path for critical workflow completes	Dependent systems are reachable (e.g., DB, Kafka)	TLS, headers, and auth are in place

Example Smoke Test Checklist

#	Check	What to Verify	Expected Result	Type
1	Application Home Page	Homepage loads (HTTP 200)	Status 200 OK	Automated
2	User Login	Login with valid user	Login successful, token issued	Automated
3	Core API Health	/api/health endpoint	Status 200 OK	Automated
4	Create Entity	Create a sample record	Record created successfully	Automated
5	Read Entity	Retrieve the created record	Record displayed correctly	Automated
6	External Dependency	Check DB/Kafka connectivity	Connectivity successful	Automated

Smoke Test Outcome

PASS	Deployment is healthy. Proceed to full testing/monitoring.
WARNING	Minor issues detected. Investigate and retest.
FAIL	Critical issue detected. Stop release and rollback.

Best Practices

- Keep the smoke test suite small and stable
- Focus only on critical paths and dependencies
- Automate and run after every deployment
- Fail fast and provide clear alerts
- Maintain tests as the application evolves

Tools & Techniques

- REST Assured / Postman / Newman
- Selenium (for UI smoke tests)
- Health check endpoints
- Custom scripts (curl, HTTP, DB checks)
- CI/CD pipeline integration

Goal: Ensure the deployed application is stable and critical functionality works before proceeding.

Result: Early detection of major issues, reduced risk, and faster, more reliable releases.

Who Benefits: Developers, QA, Operations, and Business Stakeholders.

Outcome: Higher quality deployments, fewer incidents, and better user experience.

ROLLBACK WORKFLOW

A rehearsed rollback to a previous image digest on OpenShift -- not 'just restart the pod and hope'.

ROLLBACK SEQUENCE

Trigger: a failed smoke test or a CrashLoopBackOff, with a named owner.

oc rollout undo deployment/crm-api targets a previous digest.

oc rollout status confirms the reverted deployment reaches ready.

Re-run the full smoke sequence against the rolled-back digest.

ROLLBACK DISCIPLINE

- The rollback unit is always a digest, never 'restart and hope'.
- A rollback is rehearsed on OpenShift before it is ever needed for real.
- Rollback commands and timestamps are recorded as release evidence.
- Statelessness (from Module 48) is what makes a clean rollback possible.

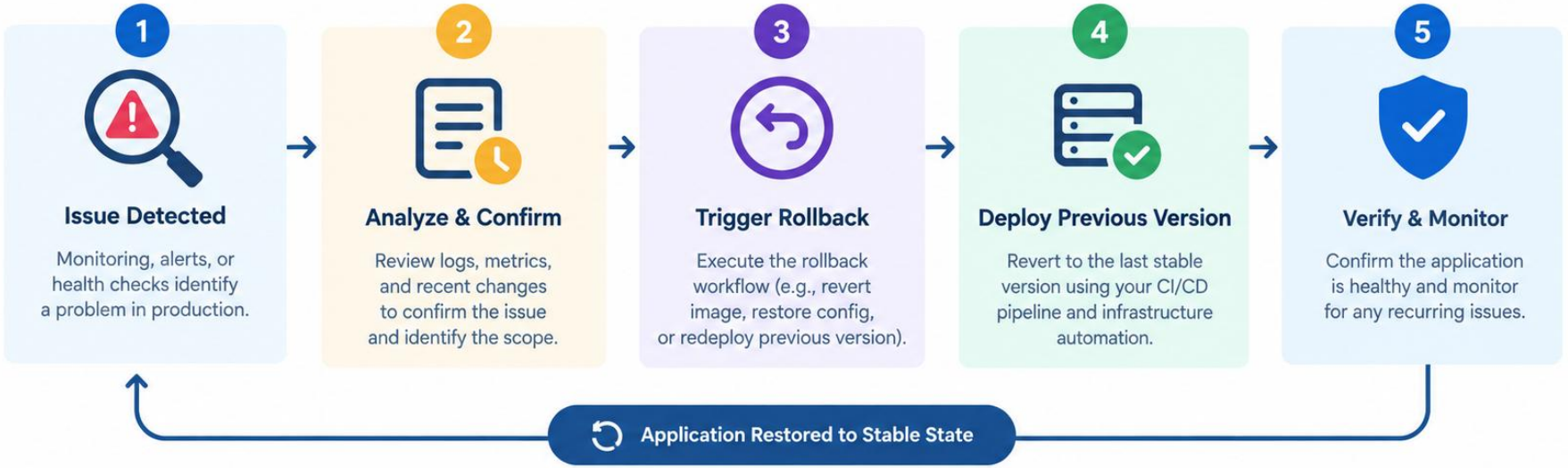
Rollback mini-runbook (concept)

```
Trigger: failed smoke or CrashLoop.   Owner: on-call lead.
oc rollout undo deployment/crm-api; oc rollout status ...; re-run smoke; confirm digest.
```

KEY TAKEAWAY A rollback plan that has never been executed against OpenShift is a hope, not evidence -- rehearse it before the Production Approval Gate, not during an incident.

Rollback Workflow

Automated rollback ensures a quick and safe recovery when issues are detected in production.



Minimizes Downtime
Restores service quickly and reduces business impact.

Automates Recovery
Uses CI/CD and infrastructure automation for consistency.

Improves Reliability
Ensures stable and predictable deployments.

Builds Confidence
Provides a clear, repeatable process for incident response.

FINAL READINESS CHECKLIST

Before applying a manifest to the shared OpenShift cluster, confirm every one of these is actually true.

Digest Pinned

The manifest references the exact scanned, published digest -- never a tag alone.

Probes Configured

Readiness and liveness probes point at the real actuator paths.

Secrets Wired

Registry pull secret and app secrets are mounted, never templated in plain text.

Context Confirmed

oc whoami and the target Project verified before oc apply runs.

KEY TAKEAWAY Skipping this checklist to save time is exactly how a CrashLoopBackOff or a wrong-project deploy becomes a live-demo surprise later.

TRY IT YOURSELF — EXERCISE 6: WRITE SMOKE, ROLLBACK AND SELF-CHECK

Capstone pre-lab gate: write the post-deploy smoke checks and the rollback runbook, then confirm the five earlier plans are genuinely complete.

STEPS (notes/lab51-smoke-rollback-readiness.md)

Step	What To Do
1 — Smoke Checks	List the post-deploy checks, including a CUS-1001 read with the correlation header.
2 — Rollback Trigger	Say what result triggers rollback and who decides.
3 — Rollback Command	Write the rollback step and how you confirm the previous version is serving.
4 — Pass Mark	Mark Pass only if all five earlier notes files are complete, not stubbed.

Smoke then rollback

```
GET /actuator/health/readiness            -> 200
GET /api/customers/CUS-1001  X-Correlation-Id: lab-request-001
any check fails -> oc rollout undo -> re-verify readiness + CUS-1001
```

TRY IT NOW — Complete Exercise 6 before Lab 51: exercises/exercise-06-smoke-rollback-readiness.md (workspace: examples/module-51-exercises).

LAB OVERVIEW — CAPSTONE SECURITY, CI/CD AND DEPLOYMENT — GITHUB ACTIONS TO OPENSHIFT

E2E Actions workflow · PR gates · Angular + Java · SAST/scan · OpenShift deploy plan · smoke/rollback |

BUSINESS SCENARIO

- Leadership freezes: No production-like OpenShift promote without Actions evidence, digest pinning, environment approval, smoke, and a written rollback. You own that gate for Northstar CRM built in Labs 49–50.

LEARNING OBJECTIVES

- Design PR vs main vs protected-environment behaviors in Actions Gate fullstack changes with Angular and Maven jobs Integrate SAST/image scan evidence into promotion criteria Describe OIDC/secrets and approval gates for OpenShift Document smoke verification and rollback

WHAT YOU BUILD

- .github/workflows/ E2E workflow(s): PR gates + main/tag path; Angular job: npm ci, test (as scoped), ng build; Java verify + SAST/dependency scan evidence (or residual risk); Container build + scan notes; digest identity

KEY TAKEAWAY This Module 51 lab completes the capstone **security and delivery spine: GitHub Actions end-to-end workflow, PR quality gates, Angular and Java build/test, SAST, container build/scan, Terraform/Ansible stages, OpenShift deploy with environments/approvals…**

LAB TASKS AND DELIVERABLES

Capstone Security, CI/CD and Deployment — GitHub Actions to OpenShift — implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Open starter/README.md.
2	Copy into java-bootcamp/examples/lab51-capstone (or extend your capstone monorepo).
3	Complete workflow TODOs: PR verify, Angular npm ci/ng build, Maven verify, placeholder scan, package-once…
4	Push a branch; capture Actions evidence under notes/screenshots/lab-51/.
5	Mark timed-path Pass criteria. Continue remaining GUIDE steps as homework.
6	.github/workflows/ E2E workflow(s): PR gates + main/tag path
7	Angular job: npm ci, test (as scoped), ng build
8	Java verify + SAST/dependency scan evidence (or residual risk)
9	Container build + scan notes; digest identity
10	Terraform/Ansible stage notes tied to Lab 48 plan
11	OpenShift deploy job/docs with environments/approvals
12	OIDC/secrets hygiene (names only in docs)

EXPECTED DELIVERABLES

- .github/workflows/ E2E workflow(s): PR gates + main/tag path Angular job: npm ci, test (as scoped), ng build Java verify + SAST/dependency scan evidence (or residual risk) Container build + scan notes; digest identity Terraform/Ansible stage notes tied to Lab 48 plan OpenShift deploy job/docs with environments/approvals OIDC/secrets hygiene (names only in docs) Smoke + rollback runbook section

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs/Evidence 10

KEY TAKEAWAY Lab 51 is graded capstone work. Pre-lab exercises 1→6 must be Pass before the lab path; keep evidence under notes/screenshots and never commit secrets.

MODULE 51 SUMMARY

In this module, we made the Northstar CRM releasable: hardened, gated, scanned, deployed to OpenShift, smoke-tested, and rollback-proven.

KEY CONCEPTS COVERED

JWT Deny-by-Default

Anonymous -> 401, wrong role -> 403, correct role -> 200 -- all tested.

GitHub Actions Gates

build -> verify -> scan -> publish -> deploy, never reordered.

Layered Scanning

SAST, DAST, dependency, secret, and image scans -- criticals fixed or exceptioned.

Immutable Images

Digest-pinned, non-root, multi-stage builds for both containers.

Real OpenShift Deploy

Every rollout, smoke test, and rollback ran against the real target.

Rollback Rehearsed

A previous-digest recovery proven before the approval gate, not during an incident.

MODULE FLOW RECAP

1. **1. Harden**
2. **2. Gate Pipeline**
3. **3. Scan & Build**
4. **4. Deploy to OpenShift**
5. **5. Prove & Rollback**

KEY TAKEAWAY No 'deployed' claim without digest identity, authenticated and deny-path smoke evidence, and a rehearsed rollback -- all four, on the real OpenShift target.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of the pipeline gate order, security scanning, and container image discipline.

1. What must happen before the deploy stage runs, per this module's fixed gate order?

A. Nothing -- deploy can run first for speed

B. build, verify, scan, and publish must all pass

C. Only a successful build is required

D. Only a human approval, with no automated gates

2. Which scan type reads Angular and Spring Boot source without running it?

A. DAST

B. SAST

C. Smoke testing

D. Load testing

3. What must a container image's runtime stage always do, per this module's Dockerfile discipline?

A. Run as root for simplicity

B. Run as a non-root user

C. Include the full Node/JDK toolchain

D. Use the :latest tag for freshness

4. What is the correct response to a critical CVE finding in a dependency scan?

A. Ignore it silently and continue

B. Fix it, or record a time-bound, owned exception

C. Wait until Module 52 to address it

D. Disable the scanner

KEY TAKEAWAY Deploy is gated behind build/verify/scan/publish; SAST reads source; runtime stages are non-root; criticals are fixed or exceptioned, never ignored.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on OpenShift deployment, GitHub Environments, smoke testing, and rollback.

5. What is this course's real deployment target, per this module?

A. The Module 42 k3s training cluster

B. OpenShift, directly -- Projects, Deployments, and Routes

C. A local Docker Compose stack only

D. Bitbucket Pipelines' own hosted runner

6. What GitHub feature enforces required reviewers before a production deploy job runs?

A. A README checklist

B. GitHub Environments protection rules

C. A Slack reminder

D. An unenforced code comment

7. Per this module's smoke testing rule, what must be proven besides the authenticated 200 response?

A. Nothing else is required

B. That an anonymous request returns exactly 401

C. That the database has zero rows

D. That the pipeline finished under one minute

8. What is the correct unit for a rollback, per this module's rule?

A. Restart the pod and hope

B. A previous, known-good image digest

C. A fresh, unrelated deploy

D. Manually editing the running pod's files

KEY TAKEAWAY OpenShift is the capstone's deployment target; GitHub Environments enforce approval; smoke must prove both 401 and 200; rollback targets a digest, never a guess.

release through gated GitHub Actions to a real OpenShift target -- with rollback rehearsed, not assumed."

MODULE 51 COMPLETE

Capstone Security, CI-CD and Deployment

You can now build a gated GitHub Actions pipeline, scan and promote a digest-pinned container, deploy it for real to OpenShift, and prove the release with authenticated and denied-path smoke evidence plus a rehearsed rollback.